INSTITUTO ARGENTINO DE LA EMPRESA

Carrera: Analista de Sistemas de Información

Asignatura: Algoritmos II

Ciclo Lectivo: 2026

TRABAJO PRÁCTICO FINAL

Sistema de Gestión de Coworking y Reservas

DOCUMENTACIÓN DEL DESARROLLO

Arquitectura en Capas, Programación Orientada a Objetos y Manejo de Errores

---

Integrantes del Equipo:

- **Tullo, Jonathan Lester** (shonimdp@gmail.com)
- **Ortega, David Samuel** (davidortegaa86@gmail.com)
- **Scollo, Maximiliano** (maximilianoscollo17@gmail.com)

Docente a cargo: Lic. Adrián Aroca

Fecha de entrega: Noviembre de 2026

Versión del documento: Versión 1.0 (Entrega Integrada Final)

ÍNDICE DE CONTENIDOS

1. Objetivo y Descripción del Sistema **3**
2. Tecnologías, Herramientas y Lenguajes Utilizados **3**
3. Organización del Trabajo en Equipo **4**
4. Arquitectura del Sistema por Capas **4**
5. Diagrama de Clases Simplificado (UML) **5**
6. Programación Orientada a Objetos y Clases C# **6**
6.1. Conexión centralizada (Conexion.cs) 6
6.2. Interfaz de disponibilidad (IReservable.cs) 6
6.3. Enumerador de estado (EstadoReserva.cs) 6
6.4. Clase base abstracta (Recurso.cs) y justificación 6
6.5. Clases derivadas especializadas (Sala, Oficina, Puesto) 7
6.6. Polimorfismo: Justificación de CalcularCosto(horas) 7
6.7. Clases de entidad (Cliente.cs y Reserva.cs) 8
6.8. Gestión en memoria con LINQ (GestoraCoworking.cs) 8
6.9. Evolución hacia el patrón DAO (GestorDAO.cs) 9
7. Manejo de Errores y Excepciones (MySQL + C#) **9**
8. Requerimientos Evolutivos (JSON y Configuración Externa) **10**
9. Instrucciones para la Ejecución del Proyecto **10**
10. Referencias Bibliográficas (APA 7.ª edición) **11**

1. Objetivo y Descripción del Sistema

Este proyecto consiste en el desarrollo de una aplicación de escritorio programada en C# con Windows Forms, conectada de manera centralizada a una base de datos MySQL alojada en el entorno de XAMPP. El objetivo principal es administrar de forma integral un espacio de coworking, permitiendo la gestión de clientes, el catálogo de recursos alquilables (salas de reunión, oficinas privadas y puestos de trabajo) y el control de reservas por fecha y horario.

El sistema resuelve los problemas operativos fundamentales del negocio: asegura que no existan reservas duplicadas para un mismo recurso en el mismo horario mediante validaciones estrictas en la base de datos (SIGNAL 45000), bloquea en la interfaz la seleccion de fechas pasadas impidiendo turnos retroactivos, calcula automaticamente los costos de alquiler aplicando tarifas base y recargos polimorficos por comodidades, y mantiene un registro automatico de auditoria para cada cancelacion.

2. Tecnologías, Herramientas y Lenguajes Utilizados

- **Lenguaje:** C# (.NET Framework 4.7.2): POO estricto, herencia, interfaces, colecciones genéricas y LINQ.
- **Interfaz:** Windows Forms (WinForms): Formularios FrmLogin y FrmPrincipal con DataGridView, ComboBox, TabControl.
- **Persistencia:** MySQL / MariaDB en XAMPP: Motor relacional en localhost:3306 con Stored Procedures, vistas, funciones y triggers.
- **Conector:** MySql.Data (v9.5.0): Conector oficial de ADO.NET para administración de conexiones y parámetros.
- **IDE:** Microsoft Visual Studio 2022: IDE de desarrollo, compilación y gestión NuGet.
- **Control de Versiones:** Git y GitHub: Control de versiones distribuido y trabajo colaborativo.

3. Organización del Trabajo en Equipo

- **Base de Datos y Núcleo:** Jonathan Lester Tullo: Base de datos MySQL completa, solución en Visual Studio, NuGet MySql.Data, clase Conexion.cs, clase abstracta Recurso.cs e interfaz IReservable.cs.
- **Lógica y POO:** Maximiliano Scollo: Clases derivadas (Sala, OficinaPrivada, PuestoTrabajo), implementación de polimorfismo en CalcularCosto, modelos Cliente y Reserva, GestoraCoworking con LINQ y diagrama DER.
- **Interfaz y Experiencia de Usuario:** David Samuel Ortega: Interfaz Windows Forms, pantalla de login FrmLogin con WhatsApp, FrmPrincipal con solapas Control, Reservas y Gestión protegida por perfil.

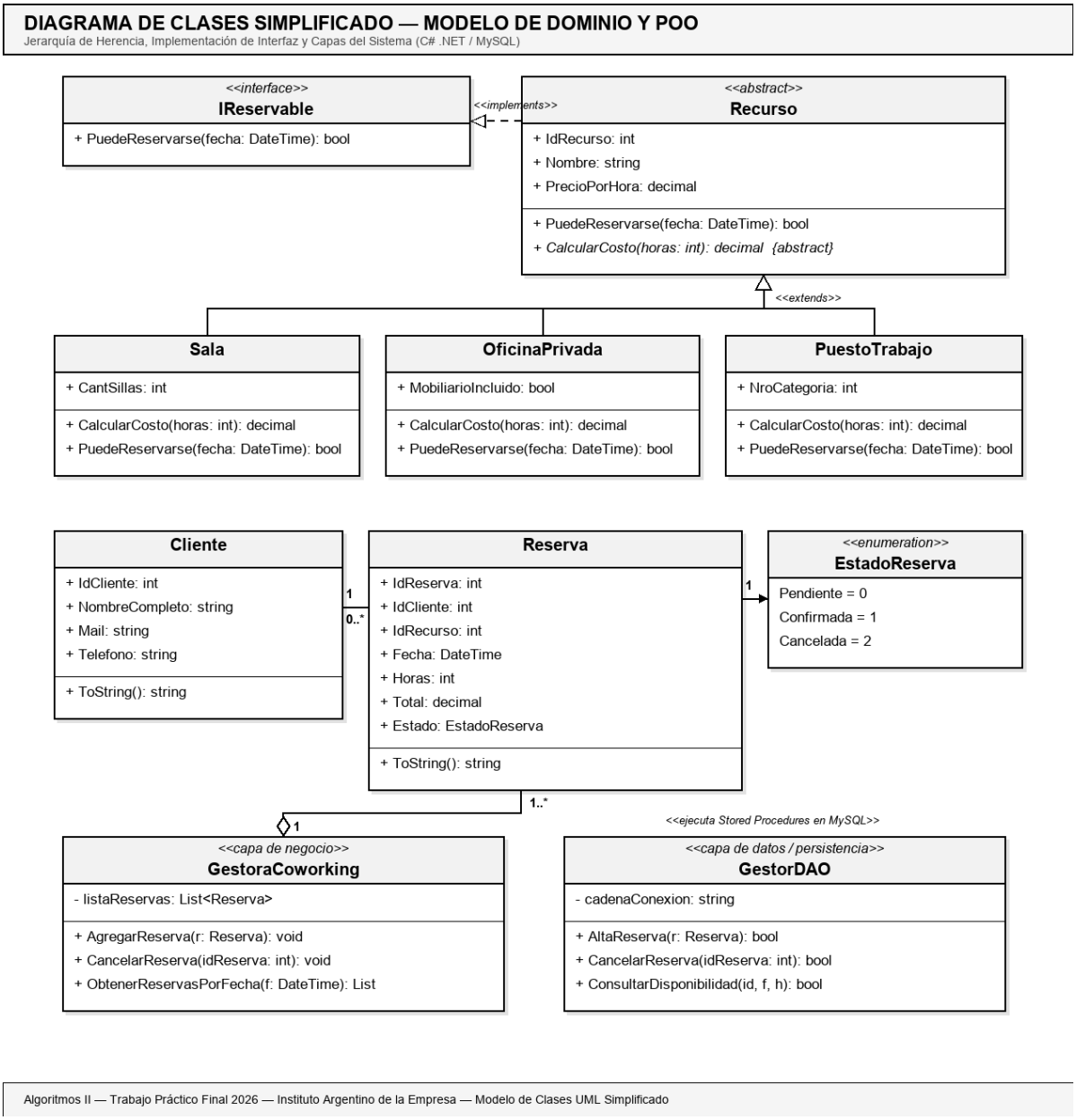
4. Arquitectura del Sistema por Capas

- **4.1. UI:** Capa de Presentación: Formularios FrmLogin y FrmPrincipal.
- **4.2. BLL:** Capa de Negocio: GestoraCoworking administra colecciones List<Reserva> en memoria con consultas LINQ.
- **4.3. Entidades:** Capa de Modelos: Clases Recurso, Sala, OficinaPrivada, PuestoTrabajo, Cliente, Reserva y EstadoReserva.
- **4.4. DAO:** Capa de Acceso a Datos: GestorDAO desacopla el consumo de procedimientos almacenados de MySQL.
- **4.5. Infraestructura:** Capa de Conexión: Conexion.cs provee instancias MySqlConnection bajo bloques using.

5. Diagrama de Clases Simplificado (UML)

Figura 1

Diagrama de Clases Simplificado del Dominio (UML)



Nota. Diagrama UML exhibiendo la jerarquía de herencia de Recurso, la interfaz IReservable y la relación con la capa DAO.

6. Programación Orientada a Objetos y Clases C#

6.1. Conexión centralizada (Conexion.cs)

Clase static con método CrearConexion(). Entrega instancias de MySqlConnection que se abren puntualmente con using, garantizando su cierre automático con Dispose().

6.2. Interfaz de disponibilidad (IReservable.cs)

Contrato formal bool PuedeReservarse(DateTime fecha). Desacopla la verificación de disponibilidad de la jerarquía de herencia.

6.3. Enumerador de estado (EstadoReserva.cs)

Enum con valores Pendiente, Confirmada, Cancelada y Finalizada, asegurando fuerte tipado.

6.4. Clase base abstracta (Recurso.cs) y justificación

No se instancian recursos genéricos porque los clientes alquilan tipos específicos de espacios. Agrupa propiedades comunes y declara el método abstracto CalcularCosto(int horas).

6.5. Clases derivadas especializadas

- **Sala:** Sala: Propiedad CantSillas. Adicional de $8.000 (>30 sillas) o $16.000 (>60 sillas).
- **OficinaPrivada:** OficinaPrivada: Propiedad MobiliarioIncluido. Canon fijo adicional de $1.500 si viene amueblada.
- **PuestoTrabajo:** PuestoTrabajo: Propiedad NroCategoria. Plus de $3.500 (cat 1), $5.500 (cat 2) u $8.000 (cat 3).

6.6. Polimorfismo: Justificación de CalcularCosto(horas)

Permite que una referencia de tipo Recurso invoque CalcularCosto dinámicamente en tiempo de ejecución sin switch ni condicionales acoplados, respetando el Principio Abierto/Cerrado (OCP).

```
public override decimal CalcularCosto(int horas)
{
    decimal costo = PrecioPorHora * horas;
    if (CantSillas > 60) costo += 16000;
    else if (CantSillas > 30) costo += 8000;
    return costo;
}
```

Código 1: Sobrescritura polimórfica en Sala.cs

6.7. Clases de entidad (Cliente.cs y Reserva.cs)

Modelan los datos del cliente y la reserva intermedia que conecta cliente, espacio y horario.

6.8. Gestión en memoria con LINQ (GestoraCoworking.cs)

Administra List<Reserva> en memoria principal mediante consultas LINQ (Where, FirstOrDefault).

6.9. Evolución hacia el patrón DAO (GestorDAO.cs)

Encapsula la creación dinámica de subclases de Recurso (Sala, OficinaPrivada, PuestoTrabajo) según el identificador de tipo, desacoplando los formularios de la lógica de instanciación directa y respetando los principios de Responsabilidad Única y Abierto/Cerrado.

6.10. Patrón Creacional Factory (RecursoFactory.cs)

Desacopla a los formularios de cualquier instrucción ADO.NET o procedimiento SQL, aplicando Responsabilidad Única.

7. Manejo de Errores y Excepciones (MySQL + C#)

Se implementa captura diferenciada de excepciones mediante MySqlException:

```
try
{
    // Ejecución de procedimiento almacenado sp_alta_reserva
}
catch (MySqlException ex)
{
    if (ex.Number == 1644 || ex.SqlState == "45000")
        MessageBox.Show("Aviso de Negocio: " + ex.Message, "Disponibilidad",
MessageBoxButtons.OK, MessageBoxIcon.Warning);
    else
        MessageBox.Show("Error Técnico MySQL: " + ex.Message, "Error",
MessageBoxButtons.OK, MessageBoxIcon.Error);
}
catch (Exception ex)
{
    MessageBox.Show("Error Inesperado: " + ex.Message, "Error General",
MessageBoxButtons.OK, MessageBoxIcon.Error);
}
```

Código 2: Captura estructurada de SIGNAL 45000 y errores generales

8. Requerimientos Evolutivos (JSON y Configuración Externa)

Se documenta el esquema JSON de intercambio para exportación de catálogos y la migración de la cadena a App.config.

9. Instrucciones para la Ejecución del Proyecto

- **1. Persistencia:** Iniciar MySQL en XAMPP (puerto 3306).
- **2. Carga BD:** Ejecutar DB/DB_TPFinal.sql en phpMyAdmin o Workbench.
- **3. Compilación:** Abrir SistemaDeGestion.slnx en Visual Studio 2022 y compilar con F5.

10. Referencias Bibliográficas (APA 7.ª edición)

Freeman, E., & Robson, E. (2020). *Head First Design Patterns* (2.ª ed.). O'Reilly Media.

Martin, R. C. (2018). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Microsoft Corporation. (2023). *Documentación de C# y .NET*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/csharp/

Oracle Corporation. (2024). *MySQL 8.0 Reference Manual*. Oracle Help Center. https://dev.mysql.com/doc/refman/8.0/en/